PBCST304

OBJECT ORIENTED PROGRAMMING

MODULE - 4

Prepared by: Aswathy.T.K

SOLID Principles in Java (https://www.javatpoint.com/solid-principles-java)

Swings fundamentals – Overview of AWT, Swing v/s AWT, Swing Key Features, Model View Controller (MVC), Swing Controls, Components and Containers, Swing Packages, Event Handling in Swings, Swing Layout Managers, Exploring Swings–JFrame, JLabel, The Swing Buttons, JTextField.

Event handling – Event Handling Mechanisms, Delegation Event Model, Event Classes, Sources of Events, Event Listener Interfaces, Using the Delegation Event Model.

Developing Database Applications using JDBC – JDBC overview, Types, Steps, Common JDBC Components, Connection Establishment, SQL Fundamentals [***For projects only***] - Creating and Executing basic SQL Queries, Working with Result Set, Performing CRUD Operations with JDBC.

SWING FUNDAMENTALS

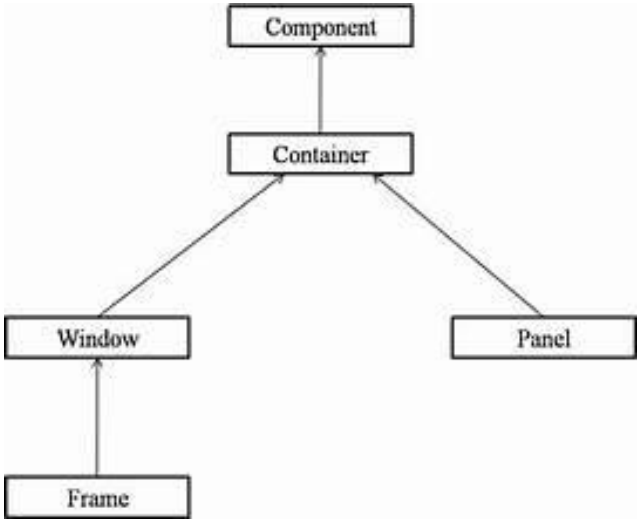
OVERVIEW OF AWT:

- AWT (Abstract Window Toolkit) is **Java's original GUI framework/toolkit used to create window-based graphical user interface (GUI) applications.**
- It can be used to create windows,buttons, menus and other user interface components.
- AWT is provided in the **java.awt package.**
- AWT is a part of **Java Foundation classes (JFC).**
- It contains numerous classes and methods to create and manage Windows.
- These classes are contained in java.awt package.
- Following are the some AWT classes:
  - AWTEvent - Encapsulates AWT events
  - Button - creates a push button control
  - Event - Encapsulates events
  - Checkbox - Creates a checkbox control
  - Frame - Generate a window that has a titile bar, menu bar and resize options.

Main features of AWT:

- Provides components for building GUI applications.
- Supports user interaction through events such as mouse clicks and keyboard actions.
- Provides layout managers to arrange components.
- Provides classes for graphics, colors, fonts, images, etc.

AWT defines windows according to a class hierarchy as shown in figure below;



- At the top of the AWT hierarchy - **Component** class.
- The **Container** is the subclass of Component.
- The **Panel** class is a concrete subclass of Container.
    - Panel is a window that does not contain a title bar, menu bar or border.
- **Window** class is a subclass of Container class.
- Frame is a subclass of Window and has a title bar, menu bar or borders and resizing options.

Swing

- Swing is a set of Java classes that **provides more powerful and flexible GUI components.**
- It is a **set of lightweight GUI components** used to develop graphical applications in Java.
- It provides the look and feel of the modern Java GUI.
- Swing is a **part of Java's GUI (Graphical User Interface) framework** used to create desktop applications with windows, buttons, text fields, menus, tables, and other graphical components.
- Swing is provided in the **javax.swing package.**

Swing - Origin / AWT pitfalls:

- Swing did not exist in the early days of Java.
- Rather, it was a response to deficiencies present in Java's original GUI subsystem: the Abstract Window Toolkit.
- The **AWT** defines a basic set of controls, windows, and dialog boxes that support a usable, but **limited graphical interface.**
- One reason for the limited nature of the AWT is that it translates its various visual components into their corresponding, platform-specific equivalents, or peers.
    - ie, the look and feel of a component is defined by the platform, not by Java.
- **AWT** components **use native code resources**. Hence, they are referred to as **heavyweight.**

- The use of native peers in AWT led to several problems.
  1. Because of variations between operating systems, a component might look, or even act, differently on different platforms.
     - This potential variability threatened the overarching philosophy of Java: write once, run anywhere.

2. The look and feel of each component was fixed (because it is defined by the platform) and could not be (easily) changed.

3. The use of heavyweight components caused many frustrating restrictions.

- Soon after Java's original release, it became apparent that the limitations and restrictions present in the AWT were sufficiently serious.
- So, a better approach was needed.
- The solution was Swing.
- Introduced in 1997, Swing was included as part of the Java Foundation Classes (JFC).

- Swing was initially available for use with Java 1.1 as a separate library.
- However, beginning with Java 1.2, Swing (and the rest of the JFC) was fully integrated into Java.
- Although Swing eliminates a number of the limitations inherent in the AWT, Swing does not replace it.
- Instead, Swing is built on the foundation of the AWT.
- This is why the AWT is still a crucial part of Java.
- Swing also uses the same event handling mechanism as the AWT.

SWING v/s AWT:**

Swing	AWT
Swing components are **not platform-dependent**	AWT components are **platform-dependent**
Swing provides several **additional components** such as scroll panes, trees etc in addition to other standard components	The AWT defines a **basic set of controls, windows, and dialog boxes** that support a usable, but limited graphical interface.
Swing is **written entirely in Java**. So swing components are **light-weight**	AWT components use native code. So they are heavy weight
Swing supports a **pluggable look and feel** (PLAF) that **can be dynamically changed** at run-time depending on environment	In AWT look and feel of **each component is fixed** and it is difficult to change its look and feel.
Swing **follow MVC**(Model View controller)	AWT **does not follow MVC**

- **Swing** is a set of classes **written entirely in Java (platform-independent).**
  - So swing components are **light-weight**
  - Swing **components are NOT implemented by platform-specific code.**
- Swing is **built on AWT**(Abstract Window Toolkit).
  - Swing extends the core functionalities of the older Abstract Window Toolkit (AWT) to **provide more powerful, flexible, and platform-independent graphical user interface (GUI) components.**
  - Swing still uses AWT for basic tasks like window creation and event handling.
- Swing provides more powerful and flexible functionalities than standard AWT components.
- Swing classes are defined in javax. swing package and its subpackages.

SWING - KEY FEATURES:**

- Swing was created to address the limitations present in the AWT.
- Following are the two key features of Swing:
  - **Lightweight components.**
  - **Pluggable look and feel.**
- Together they provide an elegant, yet easy-to-use solution to the problems of the AWT.
- More than anything else, it is these two features that define the essence of Swing.

Swing Components Are Lightweight:

- With very few exceptions, Swing components are lightweight.
- This means that, **they are written entirely in Java** and **do not map directly to platform-specific peers**.
- Because lightweight components are rendered using graphics primitives, they can be transparent, which enables non-rectangular shapes.
- Thus, **lightweight components are more efficient and more flexible.**
- Furthermore, because lightweight components do not translate into native peers, the **look and feel of each component is determined by Swing, not by the underlying operating system.**
    - This means that **each component will work in a consistent manner across all platforms.**

Native component

↓

OS controls its shape

↓

Usually rectangular

Swing lightweight component

↓

Java paints the component

↓

Java controls what pixels are painted

↓

Can have transparent areas

↓

Non-rectangular appearance is possible

Swing Supports a Pluggable Look and Feel:

- Swing supports a pluggable look and feel (PLAF).
- Because **each Swing component is rendered by Java code** rather than by native peers, **the look and feel of a component is under the control of Swing**.
- This fact means that **it is possible to separate the look and feel of a component from the logic of the component**, and this is what Swing does.
- Separating out the look and feel provides a significant advantage: -
    - It becomes **possible to change the way that a component is rendered without affecting any of its other aspects**.
    - In other words, **it is possible to "plug in" a new look and feel for any given component without creating any side effects in the code that uses that component.**

- Moreover, it becomes possible to define entire sets of look-and-feels that represent different GUI styles.
- To use a specific style, its look and feel is simply “plugged in.”
- Once this is done, all components are automatically rendered using that style.
- Pluggable look-and-feels offer several important advantages.
    - It is possible to define a look and feel that is consistent across all platforms.
    - Conversely, it is possible to create a look and feel that acts like a specific platform.
    - Eg:
        - If we know that an application will be running only in a Windows environment, it is possible to specify the Windows look and feel.
        - It is also possible to design a custom look and feel.
        - Finally, the look and feel can be changed dynamically at run time.

MODEL VIEW CONTROLLER:**

- In general, a visual component is a composite of three distinct aspects:
  - **The way that the component looks when rendered on the screen. =>VIEW**
  - **The way that the component reacts to the user. => CONTROLLER**
  - **The state information associated with the component. => MODEL**
- No matter what architecture is used to implement a component, it must implicitly contain these three parts.
- One architecture that has proven to be particularly effective for designing components is the **Model-View-Controller (MVC) architecture.**

- The **MVC architecture** is successful because **each piece of the design corresponds to an aspect of a component.**
- In MVC terminology

1.Model :

- It corresponds to the **state information associated with the component.**
- Eg: In the case of a check box, the model contains a field that indicates if the box is checked or unchecked.

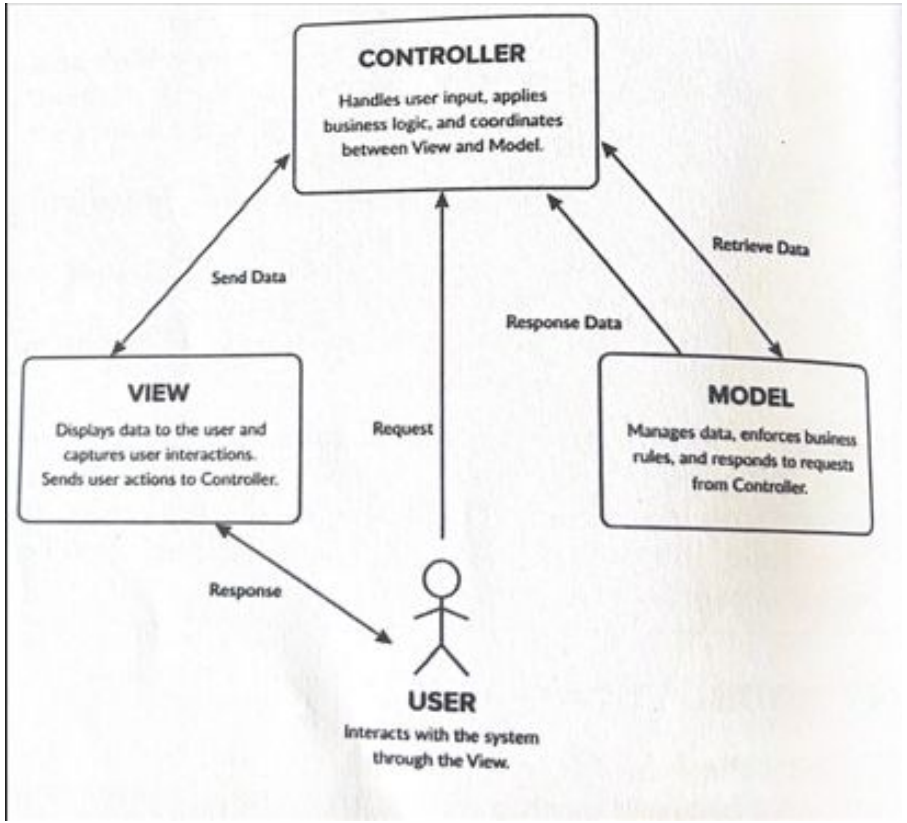
2. View :

- The view determines **how the component is displayed on the screen**, including any aspects of the view that are affected by the current state of the model.

3. Controller :

- It determines how the component reacts to the user.
- Eg: When the user clicks a checkbox, the controller reacts by changing the model to reflect the user's choice (checked or unchecked).
- This then results in the view being updated.

- **By separating a component into a model, a view, and a controller, the specific implementation of each can be changed without affecting the other two.**



1. User

- The user interacts with the system through the View.
- Eg: in a GUI application, the user may:
- Click a button
- Enter text
- Select an option

ie, The user's action generates a request.

2. View

- The View is the part that the user sees and interacts with.
- Its main responsibilities are:
  - Display data to the user
  - Accept user interactions
  - Send the user's input to the Controller

Eg: A Java Swing window containing a button and text field is part of the View.

ie, View = What the user sees and interacts with.

3.Controller:

- Acts as the intermediary between the View and Model.
- It:
  - Receives the user's request/input
  - Processes the request
  - Applies the required logic
  - Communicates with the Model
  - Coordinates the View and Model

ie, Controller = Takes the user's request and decides what should happen.

4. Model

- The Model manages the data and application logic.
  - Manages data
  - Enforces business rules
  - Performs operations requested by the Controller
  - Returns the required data/result

ie, Model = Manages the data and the rules of the application.

MVC Example for Shopping list App:

- A shopping list app is a simple application that helps users create and manage a list of items they want to buy, such as groceries or household products.
- Model (Data)
  - Stores the shopping list data.
  - Example: Item name, quantity, price, and purchased status.
  - Updates the data and notifies the view when changes occur.
- View (User Interface)
  - Displays the shopping list to the user.
  - Shows item details, total price, and purchase status.
  - Receives data from the model.
- Controller (Logic)
  - Handles user actions such as Add, Delete, or Mark as Purchased.
  - Validates input and updates the model.
  - Triggers the view to refresh with updated data.
- Eg: User taps Add Item → Controller validates the input → Model stores the new item → View refreshes and displays the updated shopping list.

SWING CONTROLS:

- Swing controls are the **GUI components used to build Swing applications.**
- A Swing application contains various **components** (controls) such as buttons, labels, text fields, check boxes, etc.
- **Components** can generate **events**, and **event listeners** are used to handle those events.
- Swing controls do not inherit features from event listeners.
- Instead, a component can register an event listener to respond to events.
- Swing controls ultimately inherit their common features through the following class hierarchy:
- Object → **Component** → **Container** → JComponent → Swing Control

Components and Containers:**

- A Swing GUI consists of two key items: **components** and **containers.**

Component:

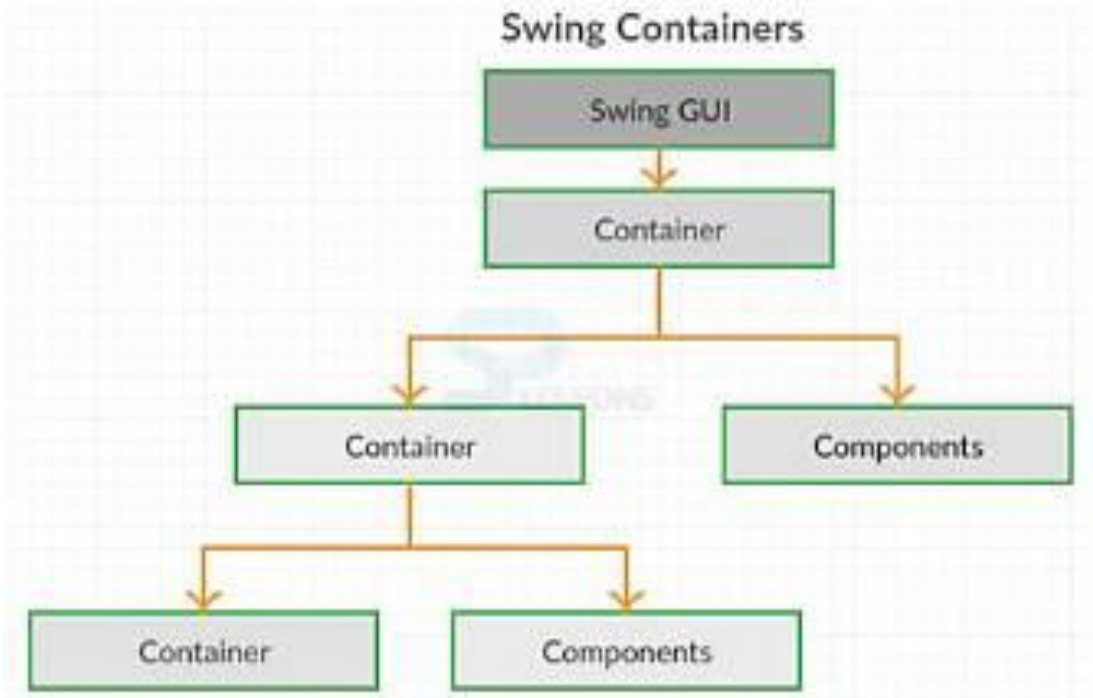
- An independent visual control used to interact with the user.
- Examples: Button, Slider, Text Field

Container:

- A special type of component that holds other components.

Note:

- All containers are components, but not all components are containers.
- **A component must be placed inside a container to be displayed.**
- **A container can hold:**
  - **Other components**
  - **Other containers**
- This arrangement creates a Containment Hierarchy.
- The top of the hierarchy must be a Top-Level Container - Eg: JFrame, JDialog, JWindow.


Swing Containers
Swing GUI
Container
Container
Components
Container
Components

COMPONENTS:

- Swing components are derived from the **JComponent class.** (Except the 4 top-level containers)
- **JComponent provides the functionality that is common to all components.**
- Eg: JComponent supports the pluggable look and feel.
  - JComponent inherits the AWT classes Container and Component.
- Thus, a Swing component is built on and compatible with an AWT component.
- All of Swing's components are represented by classes defined within the package javax.swing

Note:

- *All component classes begin with the letter J.*
- *Eg: The class for a label is JLabel; the class for a push button is JButton; and the class for a scroll bar is JScrollBar.*

- The following table shows the class names for Swing components:

JApplet	JButton	JCheckBox	JCheckBoxMenuItem
JColorChooser	JComboBox	JComponent	JDesktopPane
JDialog	JEditorPane	JFileChooser	JFormattedTextField
JFrame	JInternalFrame	JLabel	JLayeredPane
JList	JMenu	JMenuBar	JMenuItem
JOptionPane	JPanel	JPasswordField	JPopupMenu
JProgressBar	JRadioButton	JRadioButtonMenuItem	JRootPane
JScrollBar	JScrollPane	JSeparator	JSlider
JSpinner	JSplitPane	JTabbedPane	JTable
JTextArea	JTextField	JTextPane	JTogglebutton
JToolBar	JToolTip	JTree	JViewport
JWindow			

CONTAINERS:

- Swing defines two types of containers.
  - Top-level containers.
  - Lightweight containers.

Top level containers:

- These are the 4 top-level containers: **JFrame, JApplet**, **JWindow**, and **JDialog.**
- These containers **do not inherit JComponent**.
- But, **they do inherit the AWT classes Component and Container.**
- Unlike Swing's other components, which are lightweight, **the top-level containers are heavyweight.**
- This makes the top-level containers a special case in the Swing component library.

- As the name implies, a top-level container must be **at the top of a containment hierarchy.**
- A **top-level container is not contained within any other container**.
- Furthermore, **every containment hierarchy must begin with a top-level container.**
- **Examples:**
  - The one most commonly used for applications is **JFrame.**
- The one used for applets is **JApplet.**

Lightweight containers:

- Lightweight containers **inherit JComponent**
- An example of a lightweight container is **JPanel -** a general-purpose container.
- Lightweight containers are **often used to organize and manage groups of related components because a lightweight container can be contained within another container.**
- Thus, we can use lightweight containers such as JPanel to create subgroups of related controls that are contained within an outer container.

THE SWING PACKAGES:

- **Swing** is a very large subsystem and **makes use of many packages**.
- These are the packages used by Swing that are defined by Java SE 6.

javax.swing	javax.swing.border	javax.swing.colorchooser
javax.swing.event	javax.swing.filechooser	javax.swing.plaf
javax.swing.plaf.basic	javax.swing.plaf.metal	javax.swing.plaf.multi
javax.swing.plaf.synth	javax.swing.table	javax.swing.text
javax.swing.text.html	javax.swing.text.html.parser	javax.swing.text.rtf
javax.swing.tree	javax.swing.undo	

- The main package is javax.swing.
- This package must be imported into any program that uses Swing.
- It contains the classes that implement the basic Swing components, such as push buttons, labels, and check boxes.

Note:

- There are two types of Java programs in which Swing is typically used.
  - A desktop application.
  - The applet.
- Two Swing components:
  - JFrame and JLabel.
- **JFrame** is the top-level container that is commonly used for Swing applications.
- **JLabel** is the Swing component that creates a label, which is a component that displays information.
  - The label is Swing's simplest component because it is passive.
  - ie, a label does not respond to user input. It just displays output.

SWING LAYOUT MANAGERS

- Imagine designing a college application form with labels, text fields, and buttons.
- **You need to arrange these components neatly inside a window.**
- **How do we decide where each component should appear?**
  - This is where layout managers are used in Java Swing.
- **A layout manager automatically arranges components such as buttons, labels, and text fields inside a container.**
- It also adjusts their arrangement when the window is resized.

SWING LAYOUT MANAGERS:

- A layout manager **automatically arranges components such as buttons, labels, text fields, and panels inside a container.**
- Instead of specifying the exact position and size of each component, we can use a layout manager to organize them.
- **Each container has a layout manager associated with it.**
- The layout manager is set by the **setLayout( )** method.
- Syntax:

```
void setLayout(LayoutManager layoutObj)
```

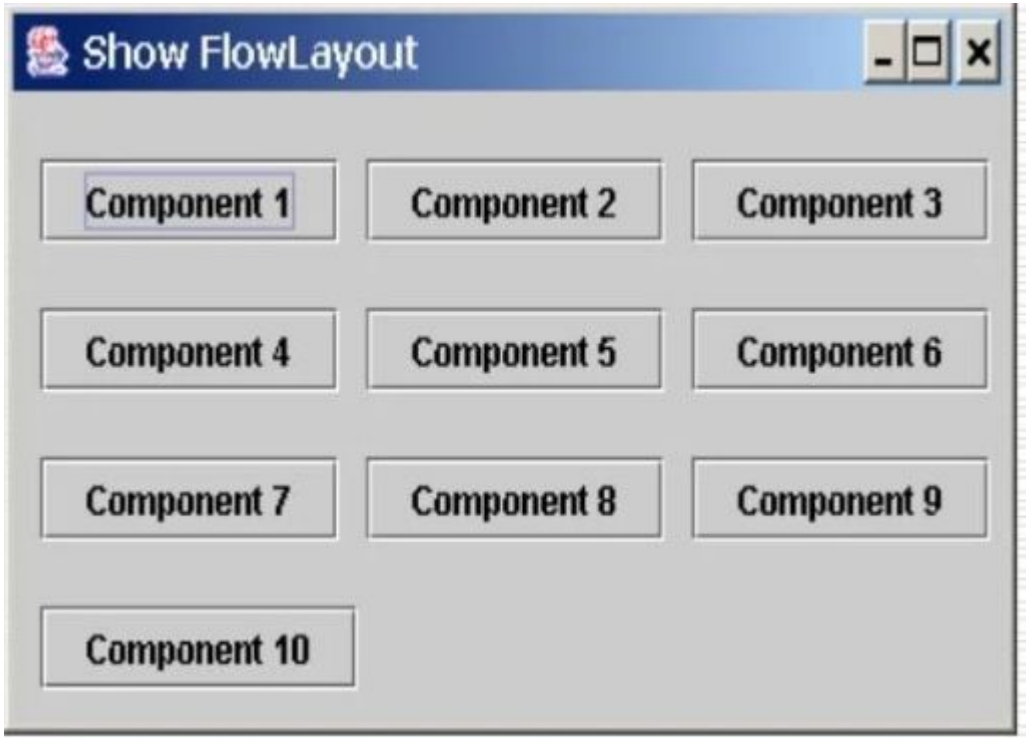
- If setLayout( ) is not called in the program, then the default layout manager is used.

- Java has several predefined LayoutManager classes;
  - **FlowLayout**
  - **BorderLayout**
  - **GridLayout**
  - **CardLayout**
  - **GridBagLayout**

FlowLayout:

- It is the **default layout manager.**
- FlowLayout has a simple layout style similar to words flow in a text editor.
- **Arranges components from left to right.**
- **Moves components to the next row when space is insufficient.**
    - **ie, the components in FlowLayout are arranged line-by-line from the upper-left corner to the next line to arrange the remaining components (left to right).**
    - **A small space is left between each component, above and below, as well as left and right.**

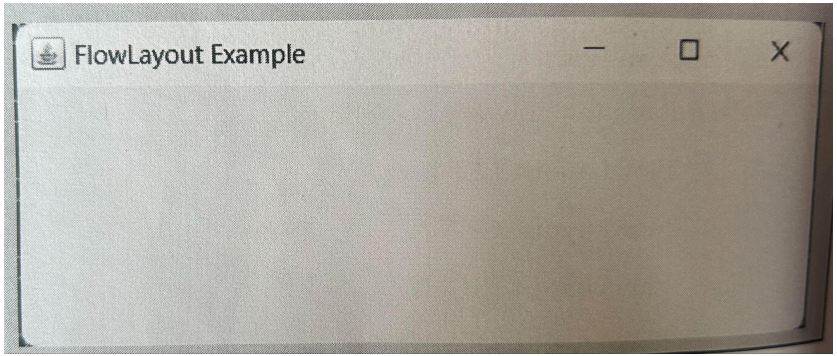
Sample flow layout:



Constructors for FlowLayout:

- **FlowLayout( )** - this creates a default layout.
  - The Components are center aligned and leaves five pixels of space between each component.
- **FlowLayout(int how)** - this specify how each line is aligned
  - how can have values:
    - FlowLayout.LEFT
    - FlowLayout.CENTER
    - FlowLayout.RIGHT
    - FlowLayout.LEADING
    - FlowLayout.TRAILING
- **FlowLayout(int how, int horz, int vert)** - in addition to other features, this also specifies the horizontal and vertical space left between components in horz and vert, respectively.

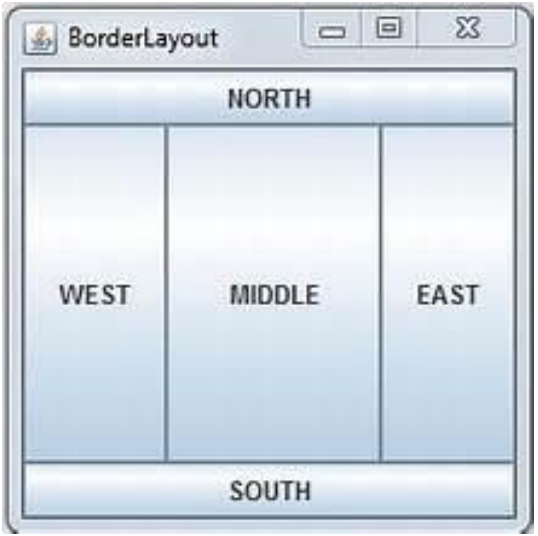
- **Write a Java swing program to create a JFrame using FlowLayout. Set the frame title as “FlowLayout Example”, the size to 300*200 pixels, and make the frame visible. Close the application when the user clicks the close button.**




```
import javax.swing.*;
import java.awt.*;
public class FlowLayout Example {
public static void main(String[ ] args) {
JFrame frame = new JFrame(“FlowLayout Example”);
frame.setLayout(new FlowLayout( ));
frame.setSize(300,200);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
} }
```

BorderLayout:

- BorderLayout is a layout manager in Java that **divides a container into five regions and arranges components in those regions.**
- The five regions are:
  - NORTH – Top
  - SOUTH – Bottom
  - EAST – Right
  - WEST – Left
  - CENTER – Middle



- BorderLayout is the default layout manager of the content pane of a JFrame.

Region	Position	Typical use
NORTH	Top	Title bar or header
SOUTH	Bottom	Status bar or footer
EAST	Right	Side panel
WEST	Left	Navigation panel
CENTER	Middle	Main content area

Constructors for BorderLayout:

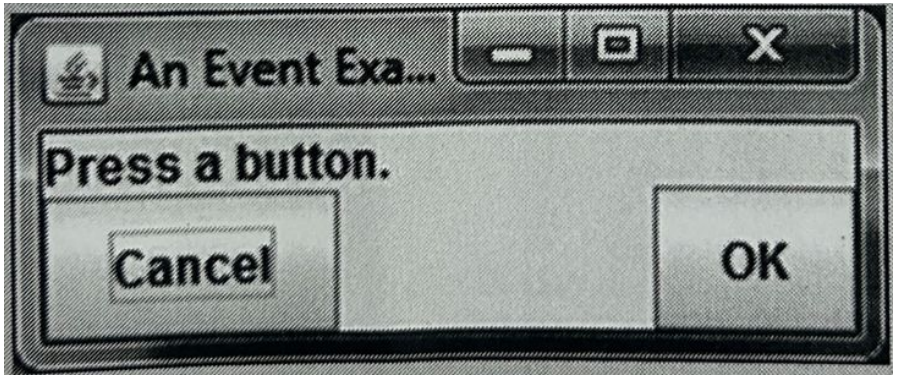
- **BorderLayout()** - Creates a BorderLayout with default gaps.
- **BorderLayout(int hgap, int vgap)** - Creates a BorderLayout with specified horizontal and vertical gaps.
  - *hgap* specifies the horizontal spacing between regions.
  - *vgap* specifies the vertical spacing between regions.

Adding components to BorderLayout:

- To place components in specific regions, use the add() method with the appropriate region constant.
- The commonly used constants are:
  - BorderLayout.NORTH
  - BorderLayout.SOUTH
  - BorderLayout.EAST
  - BorderLayout.WEST
  - BorderLayout.CENTER

Create a swing as shown:

Use Border layout.Place OK on East, Cancel on West and a label Press button on North.




```
import javax.swing.*;
import java.awt.*;
public class BorderLayout Example {
public static void main(String[ ] args) {
JFrame frame = new JFrame(“An Event  Example”); // Create frame
frame.setLayout(new BorderLayout( ));
//Create components
JLabel label = new JLabel(“Press a button”,JLabel.LEFT);
JButton okbutton = new JButton(“OK”);
JButton cancelButton = new JButton(“Cancel”);
```

```
//Add components to BorderLayout
frame.add(label, BorderLayout.NORTH);
frame.add(okButton, BorderLayout.EAST);
frame.add(cancelButton, BorderLayout.WEST);
//Frame Settings
frame.setSize(400,200);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}
```

GridLayout

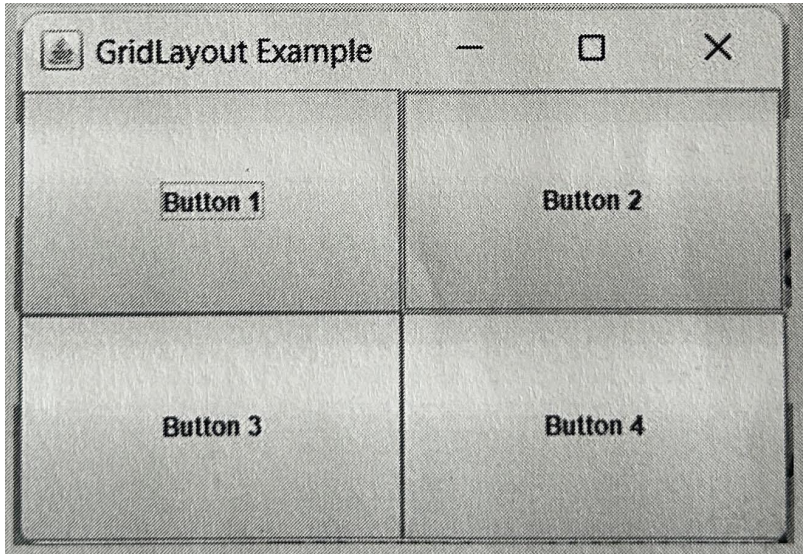
- GridLayout is a layout manager in Java that arranges components in a rectangular grid of rows and columns.
  - All components are arranged in equal-sized cells.
  - Each cell contains one component.
  - Components are placed from left to right and then from top to bottom.
  - All cells have the same width and height.
- Simple example:
  - Imagine a calculator.
  - Its buttons (0–9, +, −, ×, ÷) are arranged in rows and columns.
  - GridLayout can be used to arrange these buttons.


GridLayout Test
Button 1
Button 2
Button 3
Button 4
Button 5
Button 6

- The constructors supported by GridLayout are
  - GridLayout( )
  - GridLayout(int numRows, int numColumns)
  - GridLayout(int numRows, int numColumns, int horz, int vert)
- GridLayout ( ) - creates a single-column grid
- GridLayout(int numRows, int numColumns) - creates a grid layout with the specified number of rows and columns.
- GridLayolayoutut(int numRows, int numColumns, int horz, int vert) - allows us to specify the horizontal and vertical space left between components in horz and vert, respectively.
- Either numRows or numColumns can be zero.
- Specifying numRows as zero allows for unlimited-length columns.

Create a swing program to design the shown GUI using GridLayout.

```
import javax.swing.*;
import java.awt.*;
public class GridLayout Example {
public static void main(String[ ] args) {
JFrame frame = new JFrame(“GridLayout Example”);
frame.setLayout(new GridLayout(2,2));
frame.add(new JButton(“Button 1”));
frame.add(new JButton(“Button 2”));
frame.add(new JButton(“Button 3”));
frame.add(new JButton(“Button 4”));
frame.setSize(300,200);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
} }
```



Real-life analogy

- Think of arranging students in a classroom:
  - FlowLayout: Students sit one after another in a row.
  - BorderLayout: The classroom is divided into top, bottom, left, right, and center areas.
  - GridLayout: Students sit in equal-sized rows and columns.
  - BoxLayout: Students stand in a single horizontal or vertical line.
- Similarly, Java Swing uses different layout managers to organize GUI components.
- In short: Layout managers help us arrange GUI components automatically instead of manually specifying the position of every component.

CardLayout:

- CardLayout is a layout manager in Java that **arranges components like a stack of cards.**
- **Only one component is visible at a time, while the other components remain hidden.**
- It is **useful when we want to display different screens in the same window,** such as a login screen, registration screen, or home screen.

- Consider the following example which demonstrates the use of card layout to switch between the 3 screens (3 Cards).
  - Card 1 - Student Details
  - Card 2 - Course Details
  - Card 3 - Confirmation
- Only the card on the top is visible.



Steps:

- We must create;
  - a panel that contains the deck.
  - a panel for each card in the deck.
- Next, we add components that form each card to the appropriate panel.
- Then add these panels to the panel for which CardLayout is the layout manager.
- Finally, we add this panel to the window.
- Once these steps are complete, we must provide some way for the user to select between cards.

Constructors for card layout:

- **CardLayout( )** - Creates a CardLayout with zero horizontal and vertical gaps.
- **CardLayout(int hgap, int vgap)** - Creates a CardLayout with the specified horizontal gap (hgap) and vertical gap (vgap) between the container's edges and its components

Advantages of CardLayout:

- Displays one component at a time.

- Allows easy switching between screens.

- Reduces the need to create separate windows for each screen.

- Useful for multi-step forms, login interfaces, and wizard-style applications.

Write a Java Swing program to demonstrate CardLayout by creating a JFrame containing two cards. The first card should display a button labeled "Card 1" and the second card should display a button labeled “Card 2”.

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class CardLayoutExample implements ActionListener

{

JFrame frame;

JPanel panel;

Cardlayout card;

JButton bl, b2;
```

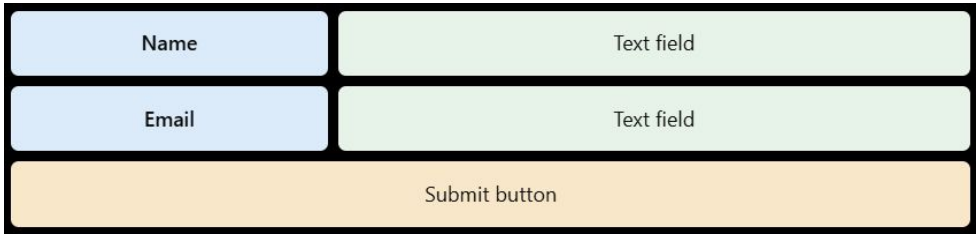
```
CardLayoutExample( ) {
frame = new JFrame(“CardLayout Example");
card = new CardLayout( );
panel = new JPanel(card);
b1 = new JButton("Card 1");
b2 = new JButton("Card 2"):
b1.addActionListener(this);
b2.addActionListener(this);
panel. add(b1, "First");
panel. add(b2, "Second");
frame.add (panel):
frame.setSize(400, 400);
frame.setDefaultCloseOperation(JFrame. EXIT_ON_CLOSE);
frame.setVisible(true);  }
```

```
public void actionPerformed(ActionEvent e)
{
String s = e.getActionLommand( );
if(s.equals( “Card 1"))
{
card.show(panel, "Second");
}
else if (s.equals(“Card 2"))
{
card.show(panel, "First");
}
}
```

```
public static void main(String[ ] args)
{
new CardLayoutExample( );
}
}
```

GridbagLayout:

- GridBagLayout is a flexible layout manager in Java AWT and Swing that **arranges components in a grid of rows and columns**.
- Unlike GridLayout, **all cells need not have the same size**, and **a component can occupy multiple rows or columns.**
- It is useful when designing forms such as login screens, registration forms, and data-entry windows.
- For example, imagine a form arranged as follows:



- Here, the Submit button spans both columns.

Steps to use a GridBagLayout

- First, create a new GridBagLayout object and to make it the current layout manager.
- Then, set the constraints that apply to each component that will be added to the grid bag.
- Finally, add the components to the layout manager.

Constructors of GridBagLayout:

- GridBagLayout defines only one constructor - **GridBagLayout( )**
- GridBagLayout defines several methods, of which many are protected and not for general use.
    - One method is setConstraints( )

      void setConstraints(Component comp, GridBagConstraints cons)

- GridBagConstraints also defines several static fields that contain standard constraint values, such as - GridBagConstraints.CENTER, GridBagConstraints. VERTICAL

Write a Java Swing program to create a Frame using GridBagLayout. Add three buttons labeled "Button 1", "Button 2" and "Button 3" in different positions using GridBagConstraints.

```
import javax.swing.*;

import java.awt.*;

public class GridBagLayoutExample {

public static void main(String[ ] args)  {

JFrame frame = new JFrame("GridBagLayout Example");

GridBagLayout layout = new GridBagLayout( );

GridBagConstraints gbc = new GridBagConstraints( );

frame.setLayout(layout);

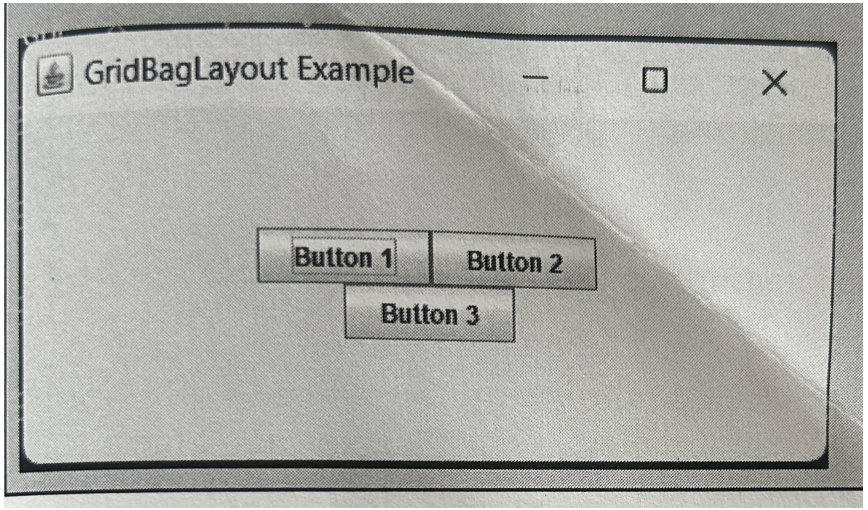
JButton b1 = new JButton("Button 1");

JButton b2 = new JButton (“Button 2”);

JButton b3 = new JButton (“Button 3”);
```

```
gbc.gridx = 0;
gbc.gridy = 0;
frame. add(b1,gbc);
gbc. gridx = 1;
gbc.gridy = 0;
frame.add(b2, gbc);
goc.gridx = 0;
gbc. gridy = 1;
gbc.gridwidth = 2; //BUTTON 3 SPANS TWO COLUMNS
frame. add(b3, gbc);
frame.setSize(300, 200);
frame setDefaultCloseperation(JFrame.EXIT_ON_CLOSE);
frame. setVisible(true); } }
```

OUTPUT



EVENT HANDLING

- Every program that uses a GUI is event-driven.
- Thus, we can't write these types of programs without a solid command of event handling.
- There are several types of events.
- Events can be generated by;
  - The mouse click(click, move, drag mouse etc.,)
  - The keyboard(type, press, release etc.,)
  - Different GUI controls, such as a
    - Push button
    - Scroll bar
    - Checkbox
- Event handling is the mechanism that controls the event and decides what should happen if an event occurs.
- Events are supported by a number of packages, including;
  - java.util
  - java.awt
  - java.awt.event

EVENT HANDLING MECHANISMS:

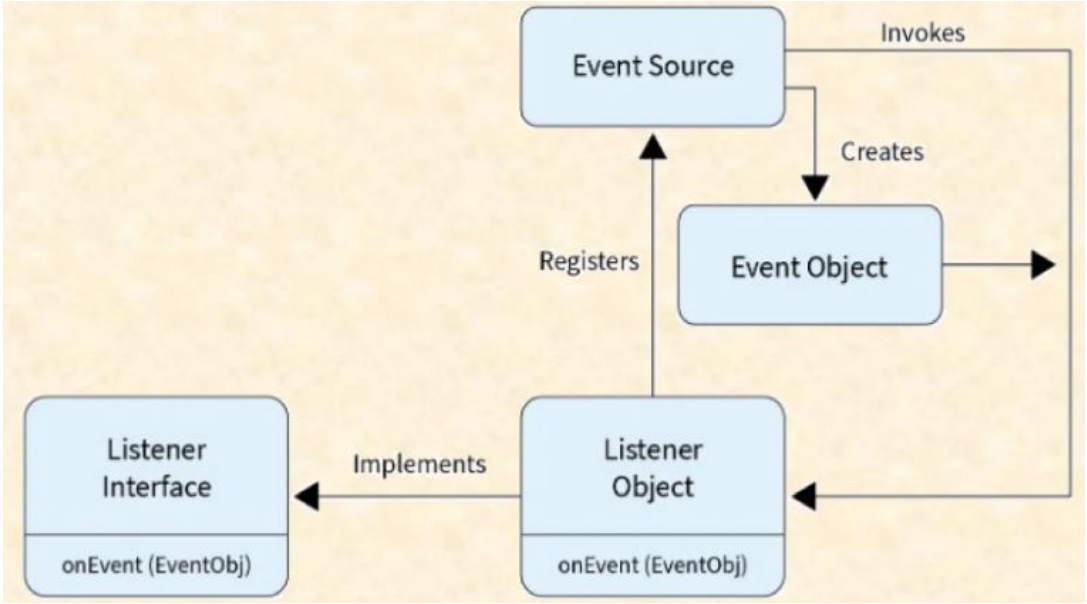
- The way in which events are handled changed significantly between the original version of Java (1.0) and modern versions of Java, beginning with version 1.1.
- The 1.0 method of event handling is still supported, but it is not recommended for new programs.
- Also, many of the methods that support the old 1.0 event model have been deprecated.
- The modern approach of event handling is based on the **delegation event model.**

DELEGATION EVENT MODEL:

- The Delegation Event Model (DEM) is the **event-handling mechanism** used in Java AWT and Swing.
- It **describes how a user action**, such as clicking a button or pressing a key, **is handled by a Java program.**
- **Three Main Components;**
  - **1. Event Source** - Object that generates an event - Eg:JButton
  - **2. Event Object** - Contains information about the event - Eg: ActionEvent
  - **3. Event Listener** - Object that receives and handles the event- Eg: ActionListener

Concept of delegation event model :

- An event source generates an event and sends it to one or more listeners.
- The listener simply waits until it receives an event.
- Once an event is received, the listener processes the event and then returns.



Advantages of Delegation Event Model:

- The application logic that processes events is cleanly separated from the user interface logic that generates those events.
- In the delegation event model, listeners must register with a source in order to receive an event notification.
- Benefit: Notifications are sent only to listeners that want to receive them.
- In previous models, an event was propagated up the containment hierarchy until it was handled by a component.
- This required components to receive events that they did not process, and it wasted valuable time. The delegation event model eliminates this overhead.

Events:

- An event is **an object that describes a state change in a source.**
- It can be **generated as a consequence of a person interacting with the elements in a GUI**.
- Some of the activities that cause events to be generated are;
  - Pressing a button
  - Entering a character via the keyboard
  - Selecting an item in a list
  - Clicking the mouse.
- Events may also occur that are not directly caused by interactions with a user interface.
  - Eg:
    - An event may be generated when a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed.
    - We can define events that are appropriate for your application.

Event Sources:

- **A source is an object that generates an event.**
- This occurs when the internal state of that object changes in some way.
- **Sources may generate more than one type of event.**
- **A source must register listeners in order for the listeners to receive notifications about a specific type of event.**
- Each type of event has its own registration method.

General form:

public void addTypeListener(TypeListener el)

- Here, Type is the name of the event, and el is a reference to the event listener.

- Eg:
  - addKeyListener( ) - The method that registers a keyboard event listener.
  - addMouseMotionListener( ) - The method that registers a mouse motion listener.
- When an event occurs, all registered listeners are notified and receive a copy of the event object.
- This is known as **multicasting** the event.
- Notifications are sent only to listeners that register to receive them.
- Some sources may allow only one listener to register.
- The general form of such a method is :

  public void addTypeListener(TypeListener el)
  throws java.util.TooManyListenersException
  - Here, Type is the name of the event, and el is a reference to the event listener.
  - When such an event occurs, the registered listener is notified.
  - This is known as **unicasting** the event.

- A source must also provide a method that allows a listener to unregister an interest in a specific type of event.
- The general form of such a method is this:

```
public void removeTypeListener(TypeListener el)
```

- Here, Type is the name of the event, and el is a reference to the event listener.
- Eg:
  - To remove a keyboard listener, call removeKeyListener( ).
- The methods that add or remove listeners are provided by the source that generates events.
- For example, the Component class provides methods to add and remove keyboard and mouse event listeners.

Event Listeners:

- A listener is **an object that is notified when an event occurs.**
- It has two major requirements.
  - It must have been registered with one or more sources to receive notifications about specific types of events.
  - It must implement methods to receive and process these notifications.
- The methods that receive and process events are defined in a set of interfaces found in java.awt.event.
- Eg:
  - The MouseMotionListener interface defines two methods to receive notifications when the mouse is dragged or moved.
  - Any object may receive and process one or both of these events if it provides an implementation of this interface.

Sources of Events

- The following table lists some of the user interface components that can generate the events.
- In addition to these GUI elements, any class derived from Component can generate events.

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu Item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Event Classes:

- The classes that represent events are at the core of Java's event handling mechanism.
- Java defines several types of events classes.
- The most widely used event classes are listed below;
  - **The ActionEvent Class**
  - **The AdjustmentEvent Class**
  - **The ComponentEvent Class**
  - **The ContainerEvent Class**
  - **The FocusEvent Class**
  - **The InputEvent Class**
  - **The ItemEvent Class**
  - **The KeyEvent Class**
  - **The MouseEvent Class**
  - **The MouseWheelEvent Class**
  - **The TextEvent Class**
  - **The WindowEvent Class**

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

- At the root of the Java event class hierarchy is EventObject, which is in java.util.
- It is the superclass for all events.
  - Its one constructor is: EventObject(Object src)
    - Here, src is the object that generates this event.
- EventObject contains two methods: getSource( ) and toString( ).
  - getSource( ) - returns the source of the event.
  - toString( ) - returns the string equivalent of the event.
- The class AWTEvent, defined within the java.awt package, is a subclass of EventObject.
- It is the superclass (either directly or indirectly) of all AWT-based events used by the delegation event model.
- Its getID( ) method can be used to determine the type of the event.

Summary:

- *EventObject is a superclass of all events.*
- *AWTEvent is a superclass of all AWT events that are handled by the delegation event model.*

The ActionEvent Class

- An ActionEvent is **generated when a button is pressed, a list item is double-clicked, or a menu item is selected.**
- The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event:
  - **ALT_MASK**
  - **CTRL_MASK**
  - **META_MASK**
  - **SHIFT_MASK.**
- In addition, there is an integer constant, **ACTION_ PERFORMED** can be used to identify action events.

- ActionEvent has these three constructors:
  - ActionEvent(Object src, int type, String cmd)
  - ActionEvent(Object src, int type, String cmd, int modifiers)
  - ActionEvent(Object src, int type, String cmd, long when, int modifiers)
- Here, src is a reference to the object that generated this event.
- The type of the event is specified by type, and its command string is cmd.
- The argument modifiers indicates which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated.
- The when parameter specifies when the event occurred.

The AdjustmentEvent Class

- An AdjustmentEvent is **generated by a scroll bar.**
- There are **five types of adjustment events.**
- The AdjustmentEvent class defines integer constants that can be used to identify them.
- The constants and their meanings are shown here:

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

- In addition, there is an integer constant, **ADJUSTMENT_VALUE_CHANGED**, that indicates that a change has occurred.
- Here is one AdjustmentEvent constructor:

```
AdjustmentEvent(Adjustable src, int id, int type, int data)
```

- Here, src is a reference to the object that generated this event.
- The id specifies the event.
- The type of the adjustment is specified by type, and its associated data is data.

The ComponentEvent Class

- A ComponentEvent is **generated when the size, position, or visibility of a component is changed.**
- There are **four types of component events.**
- The ComponentEvent class defines integer constants that can be used to identify them.
- The constants and their meanings are:

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

- ComponentEvent has this constructor:
  - ComponentEvent(Component src, int type)
    - Here, src is a reference to the object that generated this event.
    - The type of the event is specified by type.
- The getComponent( ) method returns the component that generated the event.
- It is shown here:
  - Component getComponent( )

The ContainerEvent Class

- A ContainerEvent is **generated when a component is added to or removed from a container.**
- There are two types of container events.
- The ContainerEvent class defines int constants that can be used to identify them: COMPONENT_ADDED and COMPONENT_REMOVED.
    - They indicate that a component has been added to or removed from the container.
- ContainerEvent is a subclass of ComponentEvent and has this constructor:
    - ContainerEvent(Component src, int type, Component comp)
        - Here, src is a reference to the container that generated this event.
        - The type of the event is specified by type, and the component that has been added to or removed from the container is comp.

The FocusEvent Class

- A FocusEvent is **generated when a component gains or loses input focus.**
- These events are identified by the integer constants FOCUS_GAINED and FOCUS_LOST.
- FocusEvent is a subclass of ComponentEvent and has these constructors:
  - FocusEvent(Component src, int type)
  - FocusEvent(Component src, int type, boolean temporaryFlag)
  - FocusEvent(Component src, int type, boolean temporaryFlag, Component other)
- Here, src is a reference to the component that generated this event.
- The type of the event is specified by type.
- The argument temporaryFlag is set to true if the focus event is temporary.
  - Otherwise, it is set to false.
  - Eg: Assume that the focus is in a text field. If the user moves the mouse to adjust a scroll bar, the focus is temporarily lost.)

The InputEvent Class

- The abstract class InputEvent is a subclass of ComponentEvent and is the superclass for component input events.
- Its subclasses are KeyEvent and MouseEvent.
- InputEvent defines several integer constants that represent any modifiers, such as the control key being pressed, that might be associated with the event.
- Originally, the InputEvent class defined the following eight values to represent the modifiers:

ALT_MASK	BUTTON2_MASK	META_MASK
ALT_GRAPH_MASK	BUTTON3_MASK	SHIFT_MASK
BUTTON1_MASK	CTRL_MASK	

- However, because of possible conflicts between the modifiers used by keyboard events and mouse events, and other issues, the following extended modifier values were added:

ALT_DOWN_MASK	BUTTON2_DOWN_MASK	META_DOWN_MASK
ALT_GRAPH_DOWN_MASK	BUTTON3_DOWN_MASK	SHIFT_DOWN_MASK
BUTTON1_DOWN_MASK	CTRL_DOWN_MASK	

- To test if a modifier was pressed at the time event is generated, use the isAltDown( ), isAltGraphDown( ), isControlDown( ), isMetaDown( ), and isShiftDown( ) methods.
- The forms of these methods are shown here:
  - boolean isAltDown( )
  - boolean isAltGraphDown( )
  - boolean isControlDown( )
  - boolean isMetaDown( )
  - boolean isShiftDown(

The ItemEvent Class

- An ItemEvent is generated when a check box or a list item is clicked or when a checkable menu item is selected or deselected.
- There are two types of item events, which are identified by the following integer constants:

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.

- In addition, ItemEvent defines one integer constant, ITEM_STATE_CHANGED, that signifies a change of state.
- ItemEvent has this constructor:

```
ItemEvent(ItemSelectable src, int type, Object entry, int state)
```

- Here, src is a reference to the component that generated this event.
- For example, this might be a list or choice element.
- The type of the event is specified by type.
- The specific item that generated the item event is passed in entry.
- The current state of that item is in state.

The MouseEvent Class

- There are eight types of mouse events.
- The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

- MouseEvent is a subclass of InputEvent.
- Here is one of its constructors:
  - MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)
- Here, src is a reference to the component that generated the event.
- The type of the event is specified by type.
- The system time at which the mouse event occurred is passed in when.
- The modifiers argument indicates which modifiers were pressed when a mouse event occurred.
- The coordinates of the mouse are passed in x and y.
- The click count is passed in clicks.
- The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform.

- Two commonly used methods in this class are getX( ) and getY( ).
- These return the X and Y coordinates of the mouse within the component when the event occurred.
- Their forms are shown here:
  - int getX( )
  - int getY( )
- Alternatively, we can use the getPoint( ) method to obtain the coordinates of the mouse.
  - Point getPoint( )
- It returns a Point object that contains the X,Y coordinates in its integer members: x and y.

The MouseWheelEvent Class

- The MouseWheelEvent class **encapsulates a mouse wheel event.**
- It is a subclass of MouseEvent.
- Not all mice have wheels.
- If a mouse has a wheel, it is located between the left and right buttons.
- **Mouse wheels are used for scrolling.**
- **MouseWheelEvent defines these two integer constants:**

WHEEL_BLOCK_SCROLL	A page-up or page-down scroll event occurred.
WHEEL_UNIT_SCROLL	A line-up or line-down scroll event occurred.

- Here is one of the constructors defined by MouseWheelEvent:

  MouseWheelEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup, int scrollHow, int amount, int count)

- Here, src is a reference to the object that generated the event.
- The type of the event is specified by type.
- The system time at which the mouse event occurred is passed in when.
- The modifiers argument indicates which modifiers were pressed when the event occurred.
- The coordinates of the mouse are passed in x and y.
- The number of clicks the wheel has rotated is passed in clicks.
- The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform.
- The scrollHow value must be either WHEEL_UNIT_SCROLL or WHEEL_BLOCK_ SCROLL.
- The number of units to scroll is passed in amount.
- The count parameter indicates the number of rotational units that the wheel moved.

The KeyEvent Class

- A KeyEvent is **generated when keyboard input occurs.**
- There are **three types of key events**, which are identified by these integer constants: **KEY_PRESSED, KEY_RELEASED, and KEY_TYPED.**
  - The first two events are generated when any key is pressed or released.
  - The last event occurs only when a character is generated.
- Not all keypresses result in characters.
  - Eg: Pressing SHIFT does not generate a character.
- There are many other integer constants that are defined by KeyEvent.
- For example, VK_0 through VK_9 and VK_A through VK_Z define the ASCII equivalents of the numbers and letters.

- Following are some other integer constants;

VK_ALT	VK_DOWN	VK_LEFT	VK_RIGHT
VK_CANCEL	VK_ENTER	VK_PAGE_DOWN	VK_SHIFT
VK_CONTROL	VK_ESCAPE	VK_PAGE_UP	VK_UP

- The VK constants specify virtual key codes and are independent of any modifiers, such as control, shift, or alt.
- KeyEvent is a subclass of InputEvent.
  - KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)
    - Here, src is a reference to the component that generated the event.
    - The type of the event is specified by type.
    - The system time at which the key was pressed is passed in when.
    - The modifiers argument indicates which modifiers were pressed when this key event occurred.
    - The virtual key code, such as VK_UP, VK_A, and so forth, is passed in code.
    - The character equivalent (if one exists) is passed in ch.
    - If no valid character exists, then ch contains CHAR_UNDEFINED.
    - For KEY_TYPED events, code will contain VK_UNDEFINED.

The TextEvent Class

- These are generated by text fields and text areas when characters are entered by a user or program.
- TextEvent defines the integer constant - TEXT_VALUE_CHANGED.
- The one constructor for this class is shown here:
  - TextEvent(Object src, int type)
- Here, src is a reference to the object that generated this event.
- The type of the event is specified by type.
- The TextEvent object does not include the characters currently in the text component that generated the event
- Instead, the program must use other methods associated with the text component to retrieve that information.

The WindowEvent Class

- There are **ten types of window events.**
- TheWindowEvent class defines integer constants that can be used to identify them.

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

- WindowEvent is a subclass of ComponentEvent.
- It defines several constructors.
- One such constructor is;
  - WindowEvent(Window src, int type)
- Here, src is a reference to the object that generated this event.
- The type of the event is type.
- The next three constructors offer more detailed control:
  - WindowEvent(Window src, int type, Window other)
  - WindowEvent(Window src, int type, int fromState, int toState)
  - WindowEvent(Window src, int type, Window other, int fromState, int toState)
- Here, other specifies the opposite window when a focus or activation event occurs.
- The fromState specifies the prior state of the window, and toState specifies the new state that the window will have when a window state change occurs.
- A commonly used method in this class is getWindow( ).
- It returns the Window object that generated the event.
- Its general form is shown here:
  - Window getWindow( )

EVENT LISTENER INTERFACES:

- The delegation event model has two parts - **Sources and Listeners.**
- Listeners are created by implementing one or more of the interfaces defined by the java.awt.event package.
- **When an event occurs, the event source invokes the appropriate method defined by the listener and provides an event object as its argument.**

- The following table lists the commonly used listener interfaces & provides a brief description of the methods that they define.

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The ActionListener Interface:

- This interface **defines the actionPerformed( ) method that is invoked when an action event occurs.**
- Its general form is:

```
void actionPerformed(ActionEvent ae)
```

The AdjustmentListener Interface:

- This interface **defines the adjustmentValueChanged( ) method that is invoked when an adjustment event occurs**.
- Its general form is:

```
void adjustmentValueChanged(AdjustmentEvent ae)
```

The ComponentListener Interface

- This interface **defines four methods that are invoked when a component is resized, moved, shown, or hidden.**
- Their general forms are:
  - void **componentResized**(ComponentEvent ce)
  - void **componentMoved**(ComponentEvent ce)
  - void **componentShown**(ComponentEvent ce)
  - void **componentHidden**(ComponentEvent ce)

The ContainerListener Interface

- This interface **contains two methods.**
- **When a component is added to a container, componentAdded( ) is invoked.**
- **When a component is removed from a container, componentRemoved( ) is invoked.**
- Their general forms are:
  - void componentAdded(ContainerEvent ce)
  - void componentRemoved(ContainerEvent ce)

The FocusListener Interface:

- This interface **defines two methods.**
- When a component obtains keyboard focus, **focusGained( )** is invoked.
- When a component loses keyboard focus, **focusLost( )** is called.
- Their general forms are:
  - void focusGained(FocusEvent fe)
  - void focusLost(FocusEvent fe)

The ItemListener Interface:

- This interface **defines the itemStateChanged( )** method that is invoked when the state of an item changes.
- Its general form is:
  - void itemStateChanged(ItemEvent ie)

The KeyListener Interface

- This interface **defines three methods**.
- The **keyPressed( )** and **keyReleased( )** methods are invoked when a key is pressed and released, respectively.
- The **keyTyped( )** method is invoked when a character has been entered.
- Eg:
  - If a user presses and releases the A key, three events are generated in sequence:
    - key pressed, typed, and released.
    - If a user presses and releases the HOME key, two key events are generated in sequence:
      - key pressed and released.
- The general forms of these methods are:
  - void keyPressed(KeyEvent ke)
  - void keyReleased(KeyEvent ke)
  - void keyTyped(KeyEvent ke)

The MouseListener Interface:

- This interface **defines five methods.**
- If the mouse is pressed and released at the same point, mouseClicked( ) is invoked.
- When the mouse enters a component, the mouseEntered( ) method is called.
- When it leaves, mouseExited( ) is called.
- The mousePressed( ) and mouseReleased( ) methods are invoked when the mouse is pressed and released, respectively.
- The general forms of these methods are shown here:
  - void **mouseClicked**(MouseEvent me)
  - void **mouseEntered**(MouseEvent me)
  - void **mouseExited**(MouseEvent me)
  - void **mousePressed**(MouseEvent me)
  - void **mouseReleased**(MouseEvent me)

The MouseMotionListener Interface:

- This interface **defines two methods**.
- The **mouseDragged( )** method is called multiple times as the mouse is dragged.
- The **mouseMoved( )** method is called multiple times as the mouse is moved.
- Their general forms are:
  - void mouseDragged(MouseEvent me)
  - void mouseMoved(MouseEvent me)

The MouseWheelListener Interface:

- This interface defines the **mouseWheelMoved( )** method that is invoked when the mouse wheel is moved.
- Its general form is:
  - void mouseWheelMoved(MouseWheelEvent mwe)

The TextListener Interface:

- This interface defines the **textChanged( )** method that is invoked when a change occurs in a text area or text field.
- Its general form is shown here:

```
void textChanged(TextEvent te)
```

The WindowFocusListener Interface:

- This interface defines 2 methods:**windowGainedFocus( )** and **windowLostFocus( )**.
- These are called when a window gains or loses input focus.
- Their general forms are:
  - void windowGainedFocus(WindowEvent we)
  - void windowLostFocus(WindowEvent we)

The WindowListener Interface:

- This interface **defines seven methods.**
- The **windowActivated( )** and **windowDeactivated( )** methods are invoked when a window is activated or deactivated, respectively.
- If a window is iconified, the **windowIconified( )** method is called.
- When a window is deiconified, the **windowDeiconified( )** method is called.
- When a window is opened or closed, the **windowOpened( )** or **windowClosed( )** methods are called, respectively.
- The **windowClosing( )** method is called when a window is being closed.
- The general forms of these methods are:
  - void windowActivated(WindowEvent we)
  - void windowClosed(WindowEvent we)
  - void windowClosing(WindowEvent we)
  - void windowDeactivated(WindowEvent we)
  - void windowDeiconified(WindowEvent we)
  - void windowIconified(WindowEvent we)
  - void windowOpened(WindowEvent we)

USING THE DELEGATION EVENT MODEL:

- Using the delegation event model is actually quite easy.
- Just follow these two steps:

1. Implement the appropriate interface in the listener so that it will receive the type of event desired.

2. Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.

Note:

- *A source may generate several types of events.*
- *Each event must be registered separately.*
- *Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.*
- *Two commonly used event generators: the mouse and keyboard.*